Sentence Embedding Analysis in LLMs

Eesun Moon (em3907)

In the Spring 2025 semester, I participated in the *Dynamics of Tokens in LLMs* research project at the CRIS Lab, Columbia University, under the supervision of Prof. Venkatasubramanian. This project explores the structure and behavior of sentence embeddings in large language models (LLMs), focusing on their distributional properties, clustering dynamics, and representational variation across layers and domains. By leveraging statistical laws and next-token prediction tasks, the study aims to uncover meaningful patterns in the internal representations of LLMs.

1. Zipfian Patterns in Sentence Embeddings

Understanding how sentence embeddings are structured in vector space provides a fundamental basis for analyzing token dynamics in large language models (LLMs). As an initial step in this project, I investigated whether semantically similar sentences cluster naturally in embedding space and whether such clusters follow any statistical regularities such as Zipfian patterns. To do this, I designed a pipeline to embed sentences, compute pairwise similarities, and group them based on similarity thresholds. This chapter outlines the preprocessing and similarity computation procedures, which serve as the foundation for subsequent experiments in this report.

A. Dataset and Model

For the dataset used in this chapter, I extracted 1,000 continuous sentences from the English Wikipedia dump (20231101.en) using the nltk.sent_tokenize() function. This dataset was chosen due to its diversity in topic and language structure, making it suitable for general-purpose embedding analysis. Additionally, for the LLM model to extract sentence representations, I used multiple sentence-transformer models from Hugging Face to compute sentence embeddings:

- **Small Language Model**: Sentence-transformers/all-MiniLM-L6-v2
- **Sentence BERT**: Sentence-transformers/all-mpnet-base-v2

The embeddings were computed using SentenceTransformer.encode() with default parameters.

B. Similarity-Based Grouping (Clustering)

Using sklearn.metrics.pairwise.cosine_similarity, I computed a 1000 x 1000 cosine similarity matrix between all sentence pairs. This matrix was used to assess sentence similarity, where values above 0.9 were considered “highly similar” as per the initial project instructions. However, as shown in the cosine similarity matrices below (for the first five sentences), none of the pairs exceeded the 0.9 threshold, regardless of the embedding model used. This indicates that the issue is not due to the embedding model’s scale or type but rather the nature of the dataset itself.

```
MiniLM Cosine Similarity
[[1.   0.71 0.68 0.35 0.47]
 [0.71 1.   0.49 0.44 0.36]
 [0.68 0.49 1.   0.21 0.31]
 [0.35 0.44 0.21 1.   0.4 ]
 [0.47 0.36 0.31 0.4  1.  ]]
```

```
SBert Cosine Similarity
[[1.   0.8  0.68 0.3  0.32]
 [0.8  1.   0.58 0.32 0.2 ]
 [0.68 0.58 1.   0.2  0.19]
 [0.3  0.32 0.2  1.   0.49]
 [0.32 0.2  0.19 0.49 1.  ]]
```

Figure 1. Cosine Similarity Matrices of the First Five Sentences Using MiniLM and SBERT.

Initially, each sentence was assigned to its own group, meaning no two sentences were considered similar at the 0.9 threshold. To determine whether the issue was model-specific, I repeated the process with both SBERT and MiniLM embeddings but obtained nearly identical results. Thus, I concluded that the high similarity threshold of 0.9 was too strict for this dataset. Consequently, I revised the threshold range to 0.6-0.9 and computed cosine similarity for each subsequence of the corpus. Based on this, clustering analysis was conducted iteratively using varying thresholds depending on the downstream task.

Four main grouping (clustering) strategies were implemented:

- **Greedy Grouping**: Iteratively collects all sentences within the similarity threshold.
- **Graph-based Grouping**: Constructs a similarity graph and uses connected components as clusters.
- **DBSCAN**: Density-based clustering using cosine distance (1 – similarity threshold).
- **K-Means**: Partitions the sentence embeddings into k clusters by minimizing the intra-cluster variance.

These methods allowed me to observe how embedding structures behave under different semantic grouping strategies. Unlike the other three clustering algorithms, K-means clustering requires a predefined number of clusters (k) as a hyperparameter. To automate the selection of the optimal k, I employed the elbow method with the KneeLocator algorithm. This method plots the total within-cluster sum of squares (inertia) for a range of k values and identifies the point where the rate of decrease in inertia sharply changes, forming a visible elbow in the plot. Therefore, the implementation uses the "KneeLocator" module to automatically detect the inflection point in the inertia curve, which provides a principled way to select k without relying on manual visual inspection. The corresponding k value is then selected as the optimal number of clusters, striking a balance between model complexity and cluster compactness. The pseudocodes of these four clustering methods are shown below:

```python
def grouping(cos_similar, n, thres):
  groups = []
  visited = set()

  for i in range(n):
    if i in visited:
      continue

    group = [i]
    visited.add(i)

    for j in range(i+1, n):
      if cos_similar[i][j] >= thres:
        group.append(j)
        visited.add(j)

    groups.append(group)

  return groups
```

```python
# graph-based grouping (connected componenets) - clustering
import networkx as nx

def graph_based_grouping(cos_similar, n, thres):
  graph = nx.Graph()

  graph.add_nodes_from(range(n))

  for i in range(n):
    for j in range(i+1, n):
      if cos_similar[i][j] >= thres:
        graph.add_edge(i, j)

  groups = [list(component) for component in nx.connected_components(graph)]

  return groups
```

Figure 2-1. Greedy Algorithm and Graph-based Grouping Algorithm.

```python
from sklearn.cluster import DBSCAN

def clustering_grouping(embedding, thres):
  eps = 1 - thres
  clustering = DBSCAN(eps=eps, min_samples=2, metric="cosine").fit(embedding)
  labels = clustering.labels_

  # print("The number of clusters:", len(set(labels))-(1 if -1 in labels else 0))

  groups = {}
  for idx, label in enumerate(labels):
    if label not in groups:
      groups[label] = []
    groups[label].append(idx)
  return list(groups.values())
```

Figure 2-2. DBSCAN Clustering.

```python
### Kmeans - elbow analysis
from sklearn.cluster import KMeans

def kmeans_elbow(embeddings, k_range):
    inertias = []
    for k in k_range:
        kmeans = KMeans(n_clusters=k, random_state=42, n_init='auto')
        kmeans.fit(embeddings)
        inertias.append(kmeans.inertia_)
    return inertias # within-cluster sum of squres
```

```python
def plot_elbow_with_knee(k_range, inertias, title="KMeans Elbow Plot"):
    k_vals = list(k_range)

    kl = KneeLocator(k_vals, inertias, curve="convex", direction="decreasing")
    optimal_k = kl.elbow

    plt.figure(figsize=(8, 5))
    plt.plot(k_vals, inertias, marker='o', label="Inertia")
    if optimal_k:
        plt.axvline(optimal_k, color='red', linestyle='--', label=f"Optimal k = {optimal_k}")
    plt.xlabel("Number of Clusters (k)")
    plt.ylabel("Inertia")
    plt.title(title)
    plt.legend()
    plt.grid(True)
    plt.show()

    return optimal_k
```

Figure 2-3. K-Means Clustering with Elbow Method (KneeLocator).

The number of groups formed for 1,000 sentences under different grouping algorithms is shown in the table below:

Method	MiniLM Groups	SBERT Groups
Greedy	740	654
Graph-Based	604	450
DBSCAN	65	47

Table 1. Number of Sentence Groups Formed per Clustering Method (1,000 Sentences).

C. Counting algorithms

To validate the quality and structure of the clustering results, I computed two key metrics for each group: (1) the number of sentences per group and (2) the number of unique words contained in that group. To ensure accuracy, each sentence was counted only once, and duplicate words across sentences within the same group were removed before computing word counts. These metrics served as a sanity check to identify overly large or sparse clusters that may result from suboptimal hyperparameters or flawed similarity assumptions. The following function was used to compute the sentence and word counts for each group:

Counting algorithms

```
def get_count(groups):
  group_counts = []
  word_counts = []

  for group in groups:
    group_counts.append(len(group))

    unique_words = set() # prevent duplicates
    for idx in group:
      unique_words.update(sentences[idx].split())
    word_counts.append(len(unique_words))
  return group_counts, word_counts # num of sentence per group, num of word per group
```

Figure 3. Counting Algorithm for Computing Sentence and Word Counts per Group.

The sentence and word counts for the first five randomly selected clusters across different models and grouping methods are summarized in Table 2. The results suggest that the graph-based approach tends to over-connect the initial sentences, resulting in disproportionately large clusters. Meanwhile, DBSCAN, when using min_samples=2, fails to create meaningful clusters, indicating its sensitivity to both distance thresholds and density assumptions. These observations imply that neither graph-based nor DBSCAN-based grouping is optimal within default settings for this dataset. Therefore, I selected the SBERT + Greedy combination for further analysis, based on its balanced cluster sizes and interpretable grouping behavior.

Grouping Method	Group Sizes	Word Counts
Greedy (MiniLM)	[37, 1, 1, 12, 7]	[439, 17, 12, 177, 101]
Greedy (SBERT)	[23, 1, 1, 19, 7]	[327, 17, 12, 260, 107]
Graph-Based (MiniLM)	[149, 1, 1, 1, 1]	[1362, 17, 12, 10, 35]
Graph-Based (SBERT)	[184, 1, 1, 1, 1]	[1677, 17, 12, 10, 35]
DBSCAN-Based (MiniLM)	[149, 540, 2, 2, 2]	[1362, 4452, 27, 54, 31]
DBCAN-Based (SBERT)	[184, 404, 2, 2, 2]	[1677, 3578, 21, 24, 37]

Table 2. Sentence and Word Counts for the First Five Groups across Different Clustering Methods.

D. Results: Rank-Frequency Plots, Log-Log Curves, and Linear Regression

To explore whether the semantic grouping structure follows a Zipf-like distribution, I ranked the sentence groups by their size and visualized the sentence counts:

- **Figure 4-1** shows the number of sentences per group (y-axis) plotted against the group index (x-axis), sorted in decreasing order.
- **Figure 4-2** plots the same data on a log-log scale, revealing a near-linear downward trend.
- **Figure 4-3** overlays a linear regression fit to the log-log plot, yielding a slope of -0.55 and R^2 = 0.9048, indicating a strong power-law alignment.

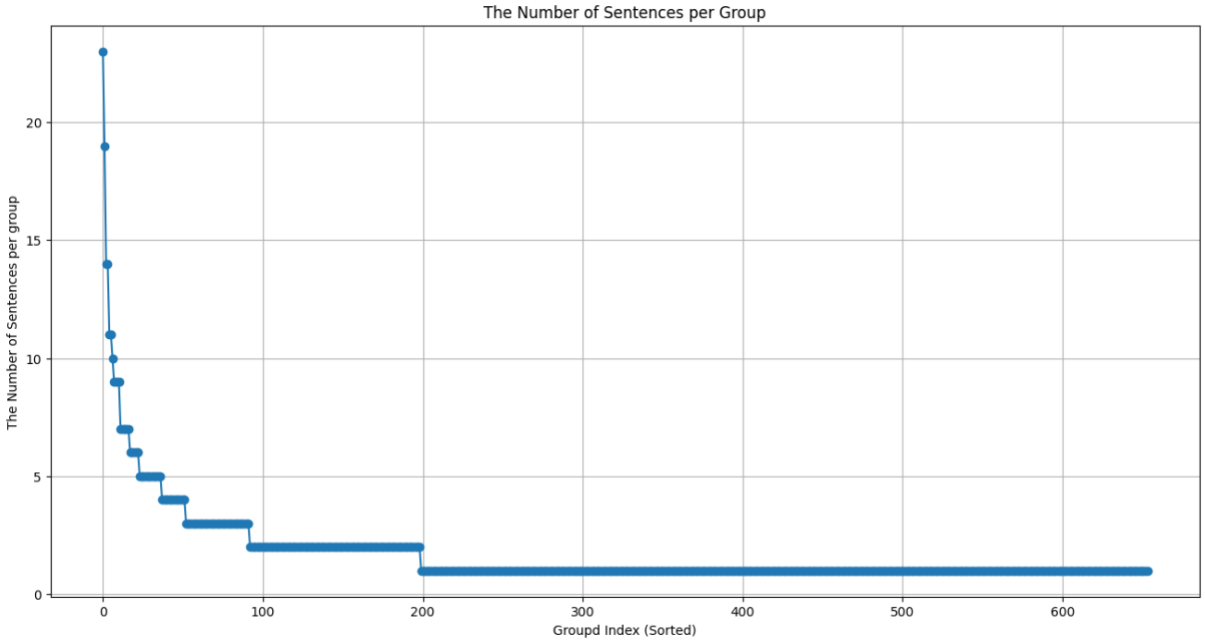


Figure 4-1. Sentence Counts Plotted against Group Index (Sorted in Descending Order).

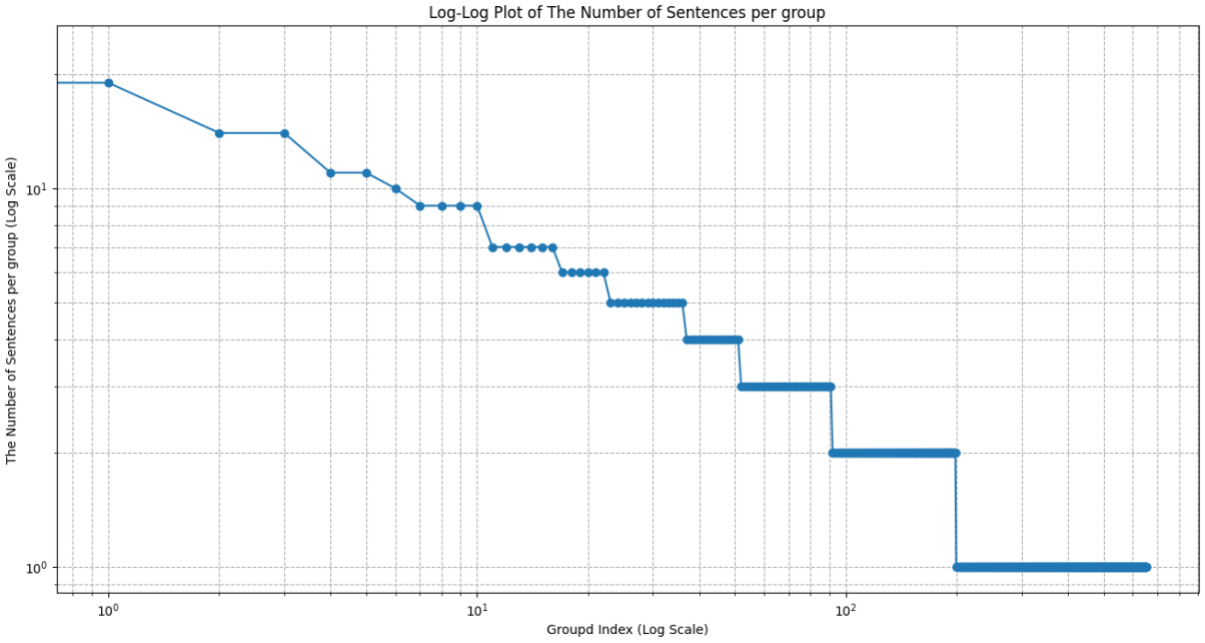


Figure 4-2. Log-Log Plot of Figure 4-1, suggesting Power-Law Behavior.

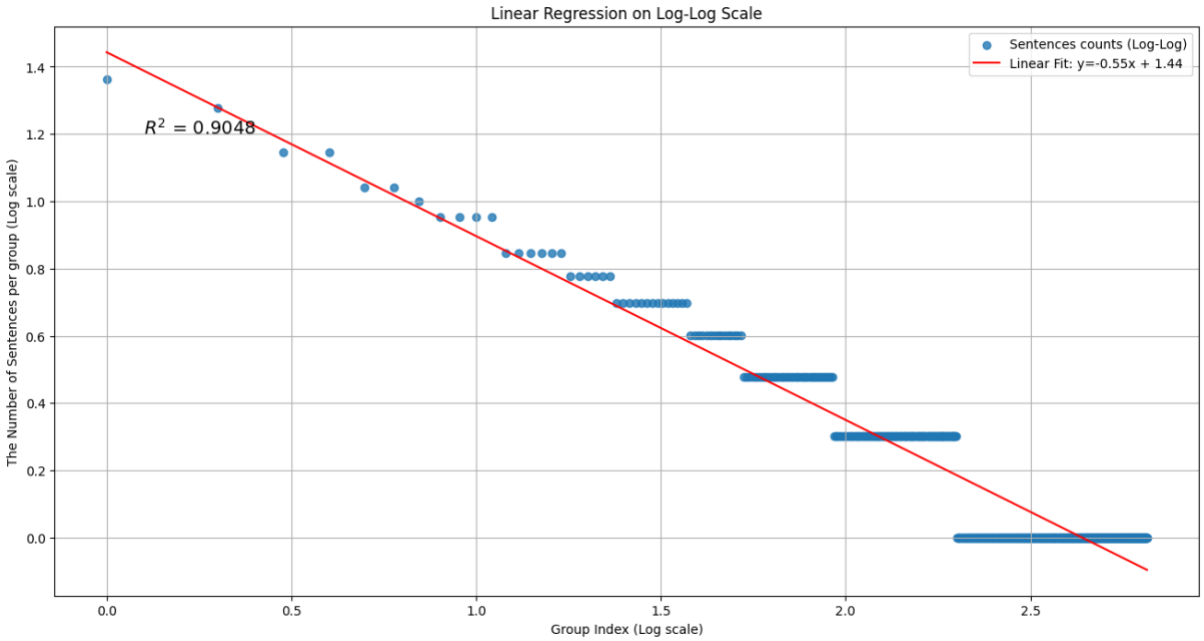


Figure 4-3. Linear regression on Log-Log Plot of Sentence Counts (R^2=0.9048).

These results support the hypothesis that sentence-level semantic similarity forms a long-tail distribution: a small number of clusters contain many semantically similar sentences, while most clusters contain only one or a few. Although the observed slope deviates from Zipf's Law (which has a slope near -1), the high R^2 value indicates that the semantic groupings do indeed follow a statistically meaningful skewed distribution.

This chapter laid the groundwork for subsequent experiments by developing a modular and reusable pipeline for sentence embedding, similarity computation, and clustering. Rather than focusing on a single dataset or task, the implementation was designed in a functional structure to ensure adaptability across domains and experimental settings. Beyond this infrastructure, the analysis also revealed key distributional behavioral patterns in LLM representations, which informed the direction of the following analyses.

In the next section, I build on these findings by analyzing how contextual depth via layer-wise representations and domain variation affect these patterns.

2. Context Vector Analysis Across Layers and Domains

While large language models (LLMs) generate token-level representations at each layer, the choice of which layer's output to use as a sentence-level context vector remains largely heuristic. In many applications, the final layer is selected by default, without rigorous analysis of how semantic properties evolve across the network. This section systematically investigates how layer depth and dataset domain influence the clustering structure of sentence embeddings.

The analysis proceeds along two axes. First, I examine whether intermediate layers, typically located closer to the middle of the model, encode semantic similarity differently than the final layer. Second, I evaluate how these effects vary across datasets that differ in topical coherence, including general-purpose corpora and thematically focused texts. In both settings, I apply consistent clustering methods and regression-based evaluation to quantify

the degree of structure in embedding space.

By analyzing slope trends in log-log plots of cluster distributions, this section aims to uncover how representational granularity shifts with both network depth and corpus characteristics, offering guidance for context vector selection in downstream tasks.

A. Layer-wise Comparison: Intermediate vs. Final Layers

To investigate how semantic grouping emerges within the internal structure of a transformer model, I begin by analyzing sentence embeddings extracted from different layers of the model. Specifically, I examine whether intermediate layers, located closer to the middle of the network, exhibit clustering behaviors that differ from those of the final output layer. By analyzing cosine similarity distributions and log-log clustering plots for both layers, this subsection aims to characterize how representational granularity evolves with depth.

To this end, I extracted embeddings from both the final and intermediate layers using the model's hidden states. For this analysis, I used a 12-layer BERT model and selected the 8th layer (i.e., the 2/3 depth point) to represent the intermediate layer. Since PyTorch indexes layers from the end using negative values, I specified layer_indices = [-1, -4] to access the final and 8th layers, respectively (see Figure 5). The embedding extraction process was implemented as a reusable batch-processing function that returned two matrices: one for each layer. These embeddings were subsequently used for clustering and similarity analysis.

```
## define the function that extracts Features

def get_features(sentence, model, tokenizer, layer_indices=[-1, -4]):
    inputs = tokenizer(sentence, return_tensors="pt", truncation=True, padding=True, max_length=128)

    with torch.no_grad():
        outputs = model(**inputs, output_hidden_states=True)  # Ensure hidden states are returned

    hidden_states = outputs.hidden_states  # Extract all hidden states

    # Extract final layer embedding (CLS token representation)
    final_layer_emb = hidden_states[layer_indices[0]][:, 0, :].squeeze(0).cpu().numpy()

    # Extract intermediate layer embedding (e.g., -4th layer)
    intermediate_layer_emb = hidden_states[layer_indices[1]][:, 0, :].squeeze(0).cpu().numpy()

    return final_layer_emb, intermediate_layer_emb
```

```
batch_size = 32
num_batches = len(sentences) // batch_size + (len(sentences) % batch_size != 0)

final_embeddings = []
intermediate_embeddings = []

for i in range(num_batches):
    batch = sentences[i * batch_size: (i + 1) * batch_size]

    # Get embeddings from model
    batch_final_emb, batch_inter_emb = zip(*[get_features(sent, model, tokenizer) for sent in batch])

    final_embeddings.extend(batch_final_emb)
    intermediate_embeddings.extend(batch_inter_emb)

final_embeddings = np.array(final_embeddings)
intermediate_embeddings = np.array(intermediate_embeddings)

print("Final embedding shape:", final_embeddings.shape)
print("Intermediate embedding shape:", intermediate_embeddings.shape)
```

Figure 5. Embedding Extraction Pipeline from intermediate and final layers using a 12-layer BERT model.

To evaluate how sentence embeddings differ across layers, I analyzed two aspects: cosine similarity distribution and clustering behavior. Figure 6 (left) shows the pairwise cosine similarity distributions of sentence embeddings from the two layers, and Figure 6 (right) compares their clustering distribution using K-Means (num_clusters=10).

The cosine similarity plot reveals that intermediate-layer embeddings (orange) exhibit higher similarity on average, mostly within the range of 0.75 to 0.9. In contrast, final-layer embeddings (blue) are more dispersed, typically ranging between 0.6 to 0.75. This suggests that final-layer representations are more diverse and specialized to individual sentence-level semantics, whereas intermediate layers produce more uniform, general-purpose embeddings that capture shared contextual structure across sentences.

The clustering comparison further supports this interpretation. Intermediate-layer embeddings are distributed more evenly across clusters, forming balanced groupings. On the other hand, final-layer embeddings are concentrated in fewer clusters, indicating a greater degree of specialization. Notably, some clusters contain significantly more embeddings from one layer than the other, suggesting that different layers encode distinct similarity patterns that influence clustering behavior.

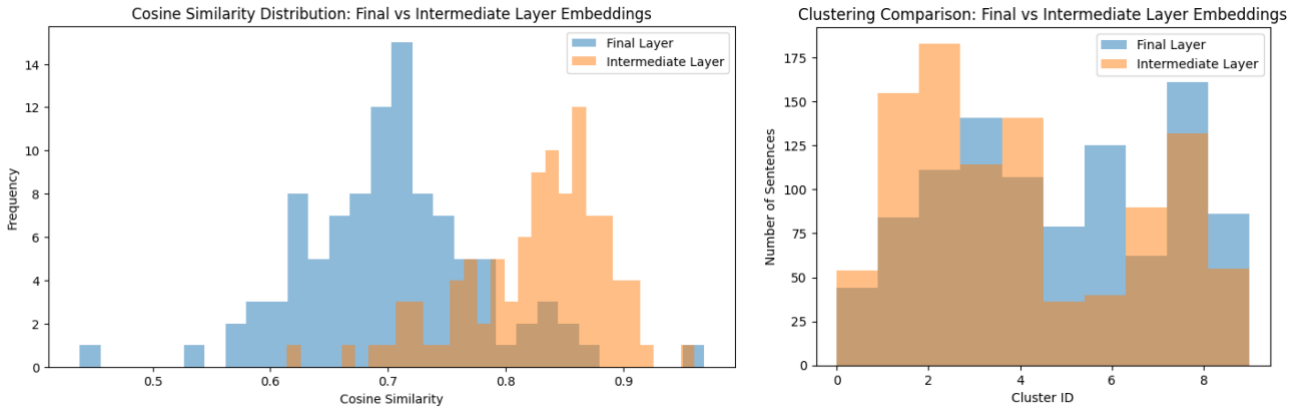


Figure 6. Left: Distribution of pairwise cosine similarity for sentence embeddings from final vs. intermediate layers. Right: K-Means-based clustering distribution comparison between final and intermediate layer embeddings.

Together, these results illustrate a trade-off between generalization and specialization across transformer layers. Specifically, intermediate layers offer more coherent, general-purpose representations that may be preferable for semantic similarity or retrieval tasks. Final layers, by contrast, encode more fine-grained and task-specific distinctions, making them better suited for classification or fine-tuning. These findings emphasize that the choice of context vector layer should be task-dependent: layers differ not just in position but in the nature of the semantic structure they encode.

B. Domain-wise Comparison: General, Romance, and Keyword-specific Corpora

While Section 2-A focused on layer-wise variation within a fixed dataset, this subsection shifts attention to another key factor: the domain of the input text. Here, I investigate whether sentence groupings become more

structured and hierarchical as the dataset becomes more thematically focused, such as in literary or keyword-specific corpora. Using consistent clustering procedures, I compare slope patterns in log-log cluster distributions across four datasets ranging from general-purpose Wikipedia to romance-focused BookCorpus subsets and analyze how domain specificity interacts with layer-wise representation.

To ensure a controlled comparison, I held all other variables constant. Sentence embeddings were extracted from a 12-layer BERT model using both the final layer and an intermediate layer (8^{th} layer, corresponding to approximately 2/3 depth). Cosine similarity was computed between sentence embeddings using a threshold of 0.9, and sentence grouping was conducted using the Greedy Clustering algorithm unless otherwise specified. The number of sentences per group was sorted in descending order and visualized on a log-log scale, followed by a linear regression fit to analyze the Zipf-like distribution.

To represent a spectrum of domain specificity, four datasets were selected:

- **Paragraph-level**: 1,000 paragraphs from the Colossal Clean Crawled Corpus
- **General-purpose**: Wikipedia (2,000 continuous sentences)
- **Romance Literary**: *Pride and Prejudice* (1,800 sentences)
- **Thematic Keywords**: BookCorpus subset (2,000 sentences containing “love” or “romance”)

All datasets were processed under identical settings, with corpus content being the only variable. This design allowed a clear evaluation of how domain granularity affects the structure of sentence embeddings across layers.

Figure 7 displays the log-log regression plots across domains and layers. In all four datasets, intermediate-layer embeddings consistently exhibited steeper slopes and higher R^2 values than final-layer embeddings, confirming their stronger clustering regularity. More notably, the slope values increased steadily as the dataset became more domain-specific:

Dataset	Final layer	Intermediate layer
C4 (paragraph-level)	-0.12	-0.87
Wikipedia (General-purpose)	-0.43	-0.89
Romance Literary (*Pride and Prejudice*)	-0.86	-1.09
Thematic Keywords (BookCorpus subset)	-1.20	-1.42

Table 3. Summary of log-log regression slopes for sentence grouping across four datasets.

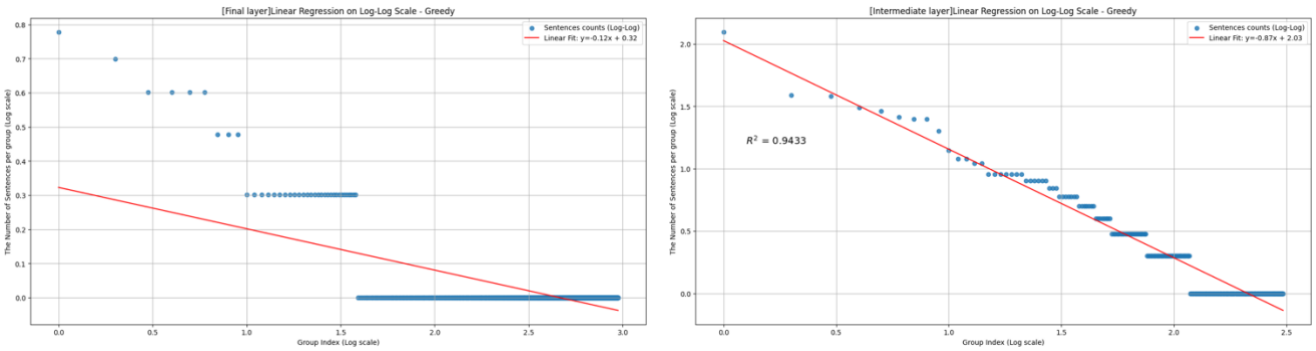


Figure 7-1. Log-log regression plots for the paragraph-level dataset (C4).
Slope = -0.12 (final layer), -0.87 (intermediate layer)

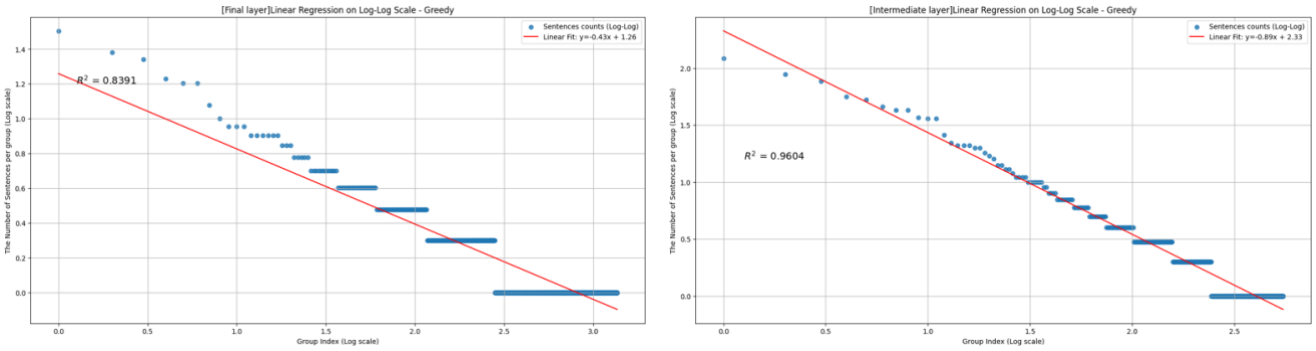


Figure 7-2. Log-log regression plots for the Wikipedia dataset.
Slope = -0.43 (final layer), -0.89 (intermediate layer)

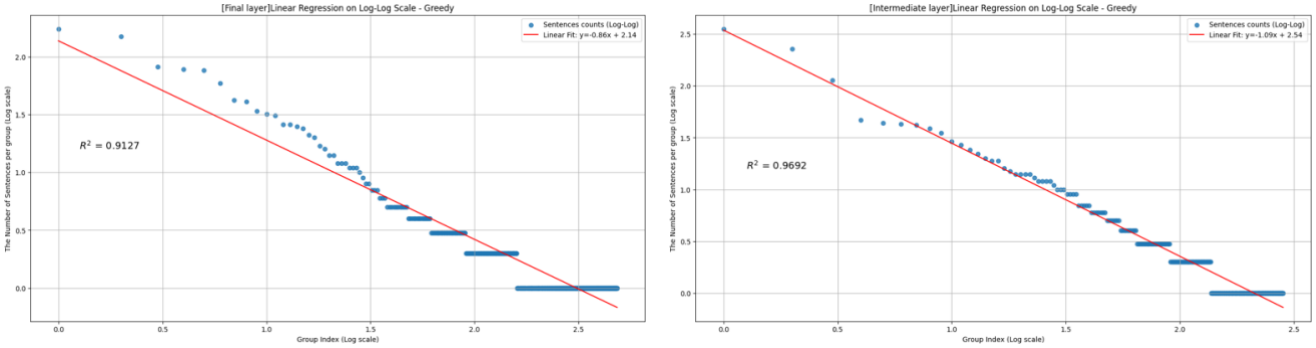


Figure 7-3. Log-log regression plots for *Pride and Prejudice*.
Slope = -0.86 (final layer), -1.09 (intermediate layer)

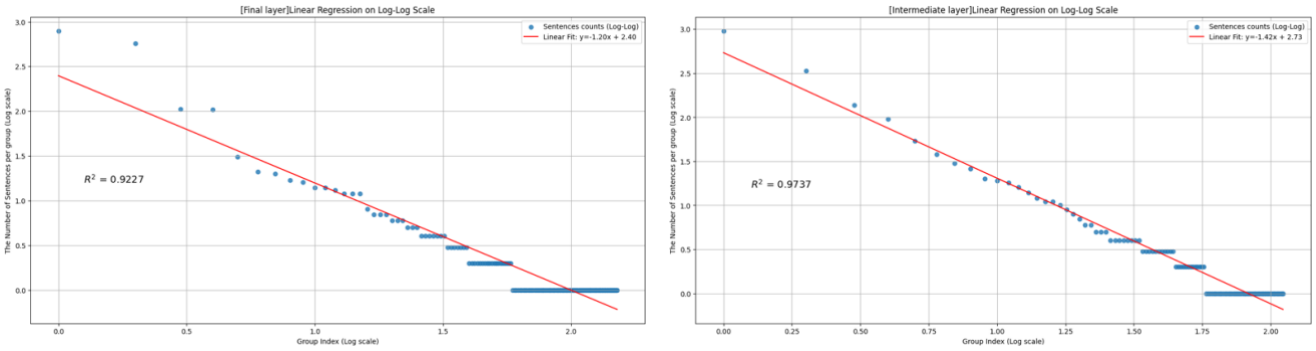


Figure 7-4. Log-log regression plots for *BookCorpus* (love/romance keywords).
Slope = -1.20 (final layer), -1.42 (intermediate layer)

This trend suggests that sentence embeddings derived from narrowly focused, topically coherent corpora, such as romantic literature or thematically filtered sentences, form stronger hierarchical groupings. In contrast, embeddings from more diverse and loosely connected text (e.g., Wikipedia or paragraph-level samples) yield flatter, less organized structures. These findings reinforce the idea that semantic grouping in embedding space is shaped by both the layer depth and the semantic specificity of the dataset. Intermediate layers are more sensitive to the internal regularities of the corpus, resulting in more Zipf-like behavior (i.e., few large clusters followed by

many small ones), while final layers appear more abstract and task-optimized.

In summary, this domain-wise comparison demonstrates that: (1) Semantic grouping becomes more structured and Zipf-like as the dataset becomes more specific and thematically coherent; (2) Intermediate layers consistently capture this structure more strongly than final layers; (3) The combined effect of domain and layer selection plays a critical role in determining how sentence representations cluster in embedding space.

3. Supplementary Clustering Analysis

A. K-Means with Elbow Method

While the previous sections compared layer-wise and domain-specific grouping structures using Greedy clustering, it is important to assess whether these trends persist across different clustering algorithms. To this end, I conducted a supplementary experiment using K-Means clustering with elbow-based hyperparameter tuning and compared the results with those of the Greedy method.

To isolate the effect of clustering strategy, the dataset was fixed to a single, domain-specific corpus: the literary work *Pride and Prejudice*. This dataset was chosen because it exhibited a steep slope and strong Zipf-like behavior in prior experiments, making it a suitable benchmark for algorithmic comparison.

Initially, three clustering algorithms were tested: (1) Greedy Clustering, (2) Graph-based Grouping, and (3) DBSCAN. All methods used a cosine similarity threshold of 0.9. As shown in Figures 8-1 and 8-2, Graph-based and DBSCAN clustering resulted in large clusters mixed with many singletons, indicating noise sensitivity and imbalance. In contrast, Greedy clustering yielded more interpretable and balanced groupings. Based on these observations, the Greedy method was retained as the primary baseline.

```
Greedy Clustering (Final Layer): 484 clusters
Greedy Clustering (Intermediate Layer): 283 clusters
Graph-based Clustering (Final Layer): 147 clusters
Graph-based Clustering (Intermediate Layer): 50 clusters
DBSCAN Clustering (Final Layer): 12 clusters
DBSCAN Clustering (Intermediate Layer): 10 clusters
```

Figure 8-1. Number of clusters formed by each method on the *Pride and Prejudice* dataset. Greedy clustering produced more balanced group counts compared to Graph-based and DBSCAN methods.

```
--Greedy Grouping--
Final layers' Group Counts of first 5th group: [11, 3, 3, 14, 78]
Intermediate layers' Group Counts of first 5th group: [228, 12, 7, 27, 2]
--Graph Grouping--
Final layers' Group Counts of first 5th group: [1641, 1, 1, 1, 1]
Intermediate layers' Group Counts of first 5th group: [1734, 2, 1, 1, 6]
--dbscan Grouping--
Final layers' Group Counts of first 5th group: [1641, 136, 2, 2, 3]
Intermediate layers' Group Counts of first 5th group: [1734, 2, 41, 6, 3]
```

Figure 8-2. Sentence count distributions for the first five clusters under each method. Graph-based and DBSCAN

cluster most sentences into a few groups, while Greedy clustering yields a more even spread.

To evaluate K-Means performance, I applied elbow analysis using the KneeLocator algorithm to determine the optimal number of clusters (k). The search range spanned 50 to 500 with a step of 10. The optimal values were: Final layer: k = 230, Intermediate layer: k = 270. As shown in Figure 9-1, the elbow points were clearly defined. Additionally, Figure 9-2 shows that sentences were distributed more uniformly across clusters compared to the highly imbalanced output of DBSCAN or Graph-based clustering.

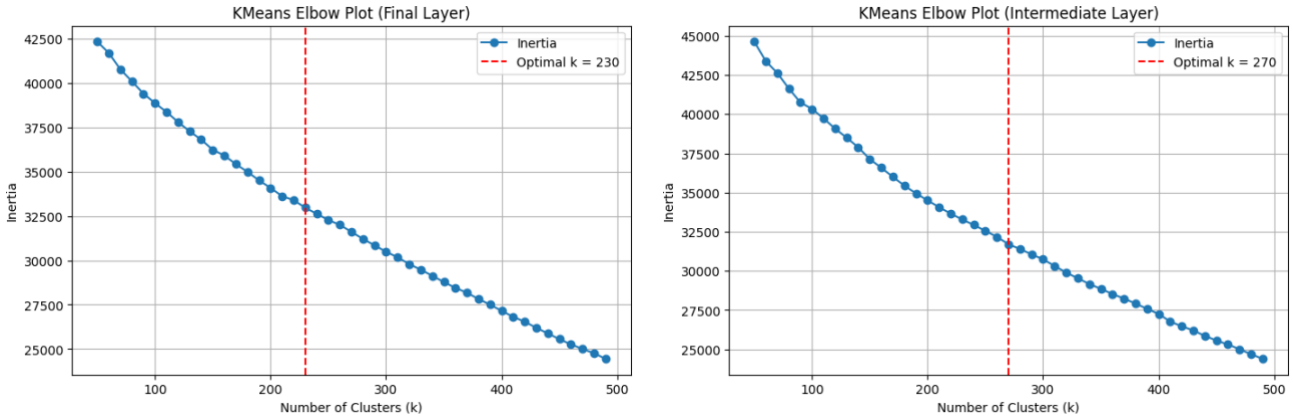


Figure 9-1. Elbow plots using the KneeLocator algorithm to identify optimal k-values for K-Means clustering. Optimal k = 230 (final layer), 270 (intermediate layer).

```
--K-means Grouping--
Final layers' Group Counts of first 5th group: [32, 30, 29, 27, 26]
Intermediate layers' Group Counts of first 5th group: [38, 32, 27, 27, 27]
```

Figure 9-2. Sentence count distributions in K-Means clustering for the first five clusters. The balanced counts suggest evenly sized group assignments.

Using the selected k values, K-Means clustering was applied to sentence embeddings from each layer. Group sizes were then sorted and visualized on log-log plots for comparison. The resulting slopes are as follows:

Clustering Method	Final layer	Intermediate layer
Greedy Algorithm	-0.86	-1.09
K-Means Clustering	-0.87	-0.93

Table 4. Clustering slope comparison between Greedy and K-Means methods.

As shown in Figure 10, both clustering methods yielded nearly identical slopes in the final layer, indicating stable grouping behavior. However, Greedy clustering yielded a steeper slope in the intermediate layer, suggesting that it more effectively captures hierarchical semantic structure.

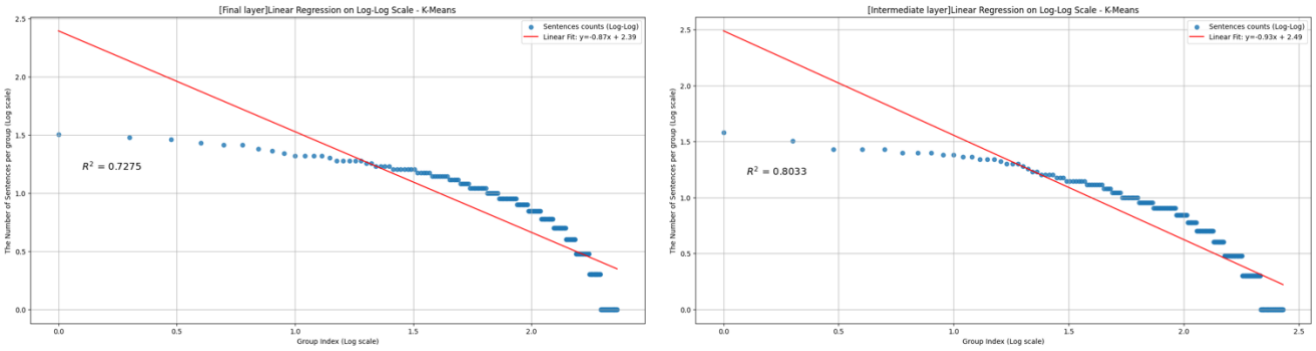


Figure 10. Log-log regression plots for *Pride and Prejudice* using K-Means.
Slope = -0.87 (final layer), -0.93 (intermediate layer)

These findings support two conclusions: First, greedy clustering remains a robust and interpretable strategy, particularly when analyzing intermediate-layer embeddings; Second, while K-Means is comparable in the final layer, its reliance on centroid-based grouping may underrepresent fine-grained semantic variation revealed in intermediate layers.

B. Token Prediction-Based Clustering

While the clustering methods explored in earlier sections, such as Greedy, Graph-based, DBSCAN, and K-Means, grouped sentence embeddings based on similarity in vector space, this subsection explores a different strategy: grouping based on predicted token identity. Instead of relying on distance metrics, each input vector is assigned to a cluster based on the token predicted by the model, offering a view into how the model's output space segments the input space.

To examine this, I used two transformer models with distinct generation mechanisms:

- **Masked BERT**, trained with masked language modeling (MLM) to predict missing tokens using bidirectional context.
- **GPT-2**, an autoregressive decoder-only model that generates the next token based on previous context.

```python
def get_features(sentence, model, tokenizer, layer_indices=-1):
    inputs = tokenizer(sentence, return_tensors="pt", truncation=True, padding=True, max_length=128)

    with torch.no_grad():
        outputs = model(**inputs, output_hidden_states=True)  # Ensure hidden states are returned

    hidden_states = outputs.hidden_states  # Extract all hidden states

    # Extract final layer embedding (CLS token representation)
    final_layer_emb = hidden_states[layer_indices][:, 0, :].squeeze(0).cpu().numpy()

    return final_layer_emb
```

Figure 11-1. Embedding Extraction using Masked BERT.

```python
def get_features(sentence, model, tokenizer, layer_indices=-1):
    inputs = tokenizer(sentence, return_tensors="pt")

    with torch.no_grad():
        outputs = model(**inputs)

    hidden_states = outputs.hidden_states[layer_indices]  # [1, seq_len, hidden_dim]

    last_token_index = -1
    final_layer_emb = hidden_states[0, last_token_index, :].cpu().numpy()

    return final_layer_emb
```

Figure 11-2. Embedding Extraction using GPT-2.

As shown in Figure 11-1 (Masked BERT) and Figure 11-2 (GPT-2), final-layer embeddings were extracted from both models. To visualize the behavior of each model across the embedding space, I projected the embeddings into 2D using PCA and overlaid a mesh grid. This mesh grid does not represent actual model inputs but serves as a visualization tool to show how different regions of the projected space correspond to different predicted tokens.

Token predictions were recorded for each mesh point. As shown in Figure 12, both models produced only three unique predicted tokens:

- **Masked BERT** predicted common stopwords such as “the”, “a”, and “.”, with “the” accounting for over 80% of the predictions. This suggests that when faced with low-information or synthetic vectors, BERT falls back on high-probability, semantically neutral outputs.
- **GPT-2**, on the other hand, produced malformed or degenerate outputs such as “C”, “.”, and “GThe”. The token “C” appeared in over 99% of predictions. This is likely due to out-of-distribution decoding behavior.

```
Total mesh points processed: 10000
Number of unique predicted tokens: 3
Top 10 most frequent predicted tokens:
the: 8270
a: 1467
.: 263
  Max predicted probability: 0.0806
  Min predicted probability: 0.0172
  Average predicted probability: 0.0368
```

```
Total points processed: 1000
Number of unique predicted tokens: 3
Top 10 most frequent predicted tokens:
Ċ: 996
.: 3
ĠThe: 1
  Max predicted probability: 0.3048
  Min predicted probability: 0.0789
  Average predicted probability: 0.2102
```

Figure 12. Token Prediction-Based Clustering Results (Left: Masked BERT, Right: GPT-2).

These results illustrate that both models reduce a large region of the embedding space to a few dominant tokens, but in different ways. BERT tends to fall back on high-frequency, semantically neutral words, likely due to its MLM training objective. GPT-2, by contrast, often generates structurally invalid tokens when confronted with out-of-distribution vectors, revealing a failure mode of autoregressive models under unconstrained inputs.

To further examine the structure of the embedding space, I projected the original embeddings and mesh grid points into PCA space (Figure 13). However, since PCA reduces dimensionality, this projection cannot faithfully preserve semantic relationships. Similarly, Figure 14 visualizes predicted token assignments (Masked BERT) across the mesh grid. The resulting clusters appear clean and rectangular, but this structure is an artifact of the PCA axes rather than true semantic separability. However, one meaningful observation emerges: as shown in Figure 14, most mesh points fall into a single cluster with the predicted token “the”. This suggests that even sentence embeddings are often grouped together under the same generic token label. The clustering, in this form, lacks the granularity needed to distinguish meaningfully different inputs. This highlights a limitation of token prediction-based clustering: despite offering a glimpse into the model’s output behavior, it fails to uncover nuanced or hierarchical semantic structure.

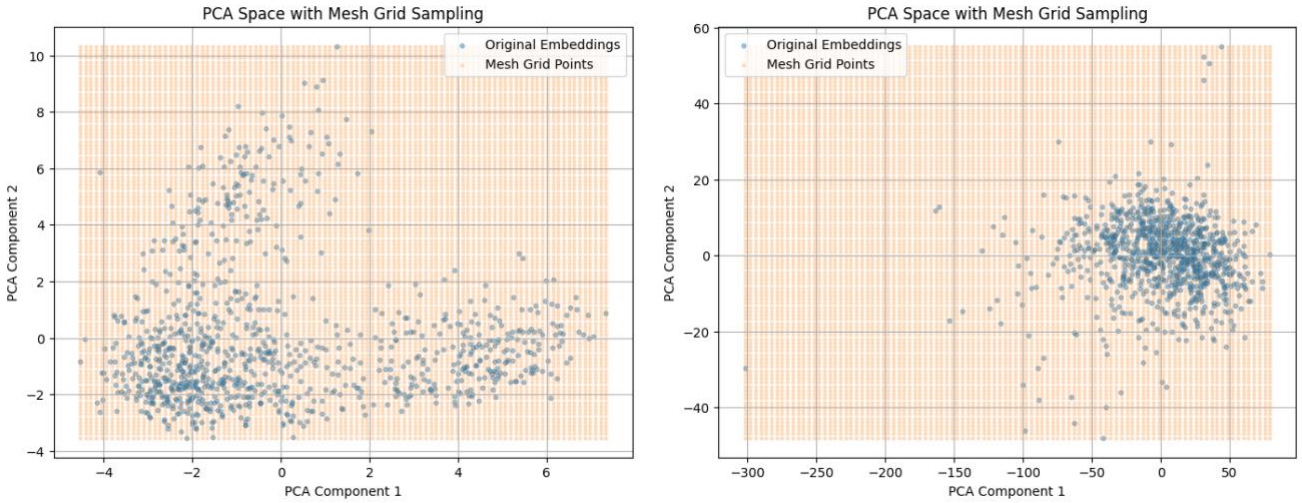


Figure 13. PCA Projection of Embedding Space with Mesh Grid Sampling (Left: Masked BERT, Right: GPT-2).

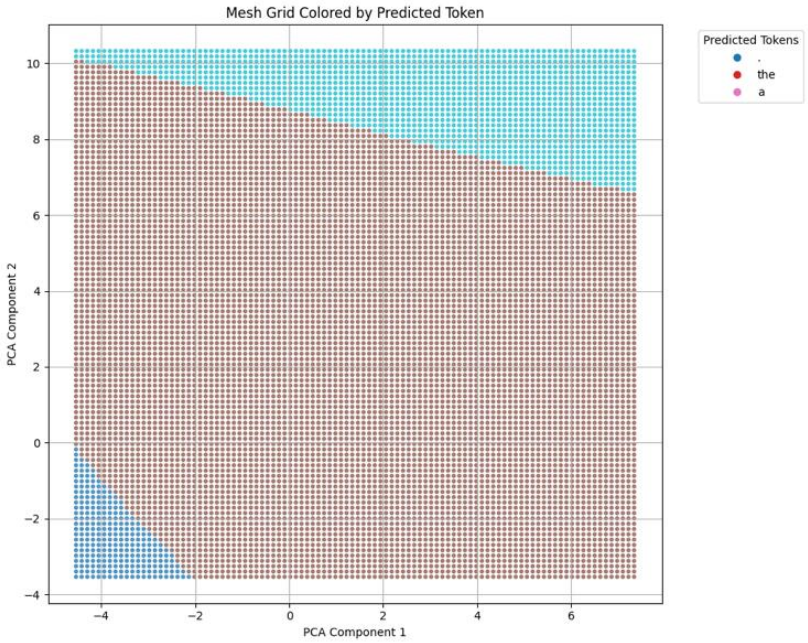


Figure 14. Mesh Grid Colored by Predicted Token (Masked BERT).

This analysis leads to two key takeaways: First, in terms of coarse partitioning, token prediction-based clustering collapses much of the input space into a few dominant token regions, limiting its ability to reveal fine-grained or hierarchical semantic groupings. Second, in the context of the architecture-specific failure modes, BERT falls back on stopwords when given synthetic or uninformative vectors, and GPT-2 generates malformed or invalid tokens when faced with vectors outside its training distribution, exposing vulnerabilities in autoregressive generation.

In summary, while clustering based on token prediction is conceptually intuitive, it does not expose rich semantic structure in the embedding space. Instead, it highlights the limits of generalization in LLMs when exposed to unfamiliar or interpolated inputs. This motivates the next chapter, which introduces hierarchical clustering methods to explore whether finer-grained substructure exists within coarse output-based clusters.

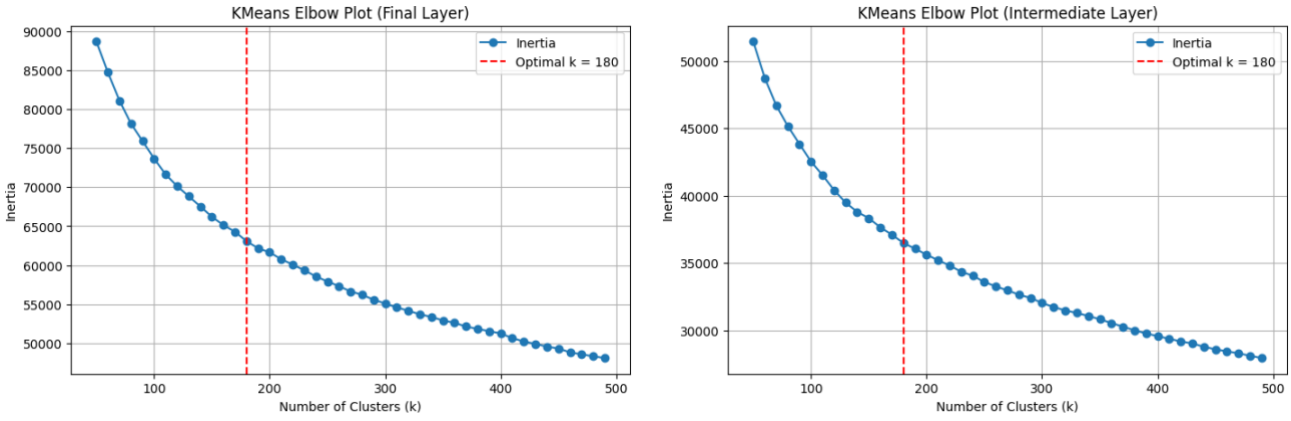
4. Unpacking Clusters: A Histogram-Based Analysis of Intra-Cluster Structure

While previous chapters focused on clustering sentence embeddings based on similarity or token prediction, this chapter shifts the focus to the internal structure of clusters. Specifically, I ask: Are clusters topically homogeneous? Can further substructure be uncovered at a finer granularity?

To explore this, I apply my analysis to two datasets of 10,000 sentences each:

- A general-purpose corpus (Wikipedia),
- A thematic corpus (BookCorpus subset with sentences containing “love” or “romance”).

Using the same pipeline from Chapters 1-2, I extracted embeddings from both final and intermediate layers and applied K-Means clustering. The optimal number of clusters (k=180) was determined using the elbow method and was consistent across both datasets and both layers (Figure 15). This consistency may result from the increased dataset size (~10,000 vs. ~1,000 previously), providing a more stable clustering foundation. Figure 16 represents the number of sentences in each cluster after applying the K-Means clustering method.



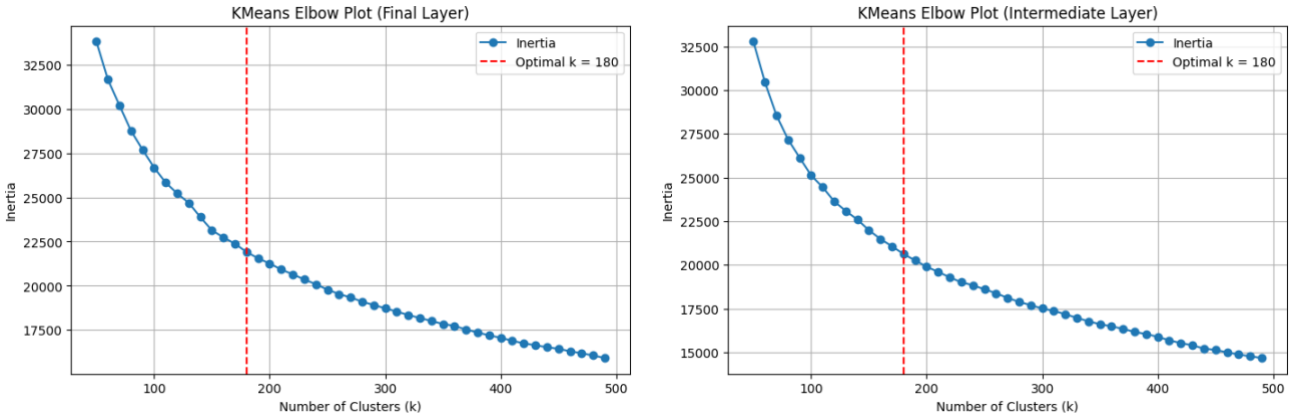


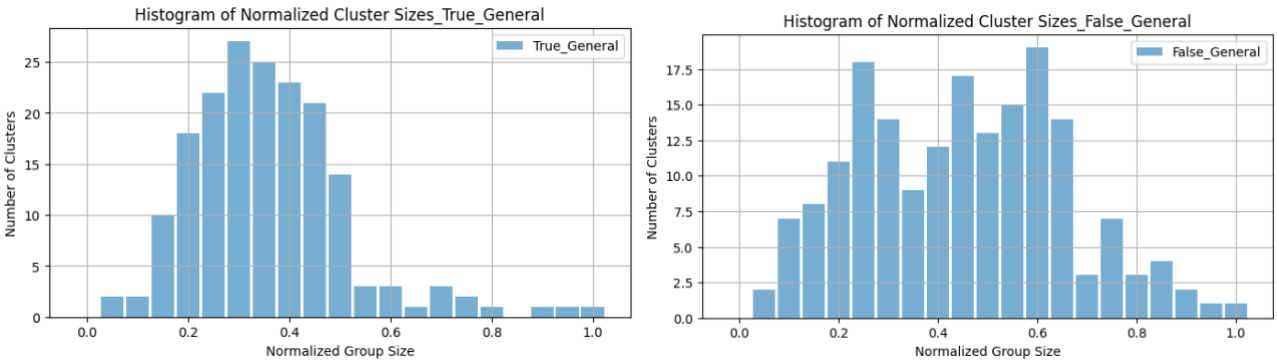
Figure 15. Elbow plots identifying the optimal number of clusters (k=180) for general-purpose (top) and love-specific datasets (bottom).

```
--K-means Grouping--
General
Final layers' Group Counts: [166, 151, 142, 129, 124]
Intermediate layers' Group Counts: [131, 118, 113, 112, 109]
Love
Final layers' Group Counts: [1167, 112, 105, 104, 102]
Intermediate layers' Group Counts: [1167, 164, 159, 159, 157]
```

Figure 16. Sentence counts of top clusters after K-Means clustering across datasets and layers.

After clustering, I computed the number of sentences in each cluster and normalized the counts by dividing each by the maximum cluster size. These normalized values were binned into 20 equally spaced intervals (e.g., [0.00 – 0.05], [0.05 – 0.10], …). The resulting histograms in Figure 17 reveal key differences:

- The general-purpose dataset is distributed more evenly across bins.
- The love-specific dataset concentrates heavily in just 2-3 bins, indicating skewed cluster sizes and potential semantic saturation.



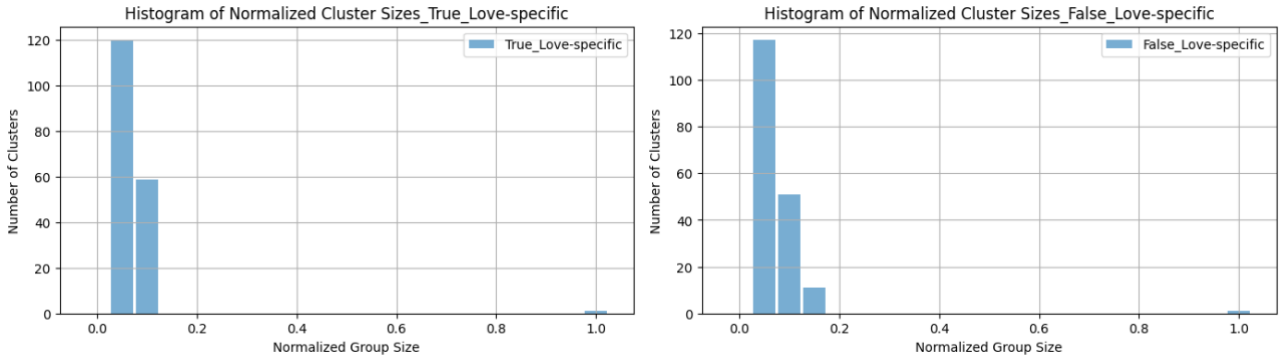


Figure 17. Histogram of normalized cluster sizes (Left: Final layer, Right: Intermediate layer). General-purpose is broadly distributed; Love-specific is skewed.

To better understand the sentence composition within each bin, I systematically selected representative clusters from specific bins and examined five sample sentences from each. For the general-purpose dataset, I analyzed final-layer embeddings and chose two clusters from bins [0.40, 0.45] and [0.45, 0.50], which correspond to cluster 31 (76 sentences) and cluster 17 (80 sentences), respectively. Among the 21 clusters in bin [0.40, 0.45] and 14 clusters in bin [0.45, 0.50], these were selected as representatives for the general-purpose dataset. For the love-specific dataset, I focused on intermediate-layer embeddings, given that the dataset is already thematically coherent. I selected cluster 12 (39 sentences) from bin [0.05, 0.10] and cluster 1 (78 sentences) from bin [0.10, 0.15]. In each case, five sentences were sampled and visualized in Figures 18-1 and 18-2. The choice of layer for each dataset reflects a practical consideration: while the general-purpose dataset benefits from the more semantically refined final-layer embeddings, the love-specific dataset already exhibits thematic consistency and is therefore better analyzed using intermediate-layer representations to observe generalization behavior.

```
-- Cluster 31 (size=76) --
• The score to An American in Paris was scheduled to be issued first in a series of scores to
be released.
• The entire project was expected to take 30 to 40 years to complete, but An American in Pari
s was planned to be an early volume in the series.
• Reception
In Movie - Film - Review, Christopher Tookey wrote that though the actresses were "competent
in roles that may have some reference to their own careers", the film "is visually unimaginat
ive, never escapes its stage origins, and is almost totally lacking in revelation or surprisi
ng incident".
• Noting that there were "occasional, refreshing moments of intergenerational bitchiness", th
ey did not "justify comparisons to All About Eve", and were "insufficiently different to dese
rve critical parallels with Rashomon".
• He also wrote that The Guardian called the film a "slow, stuffy chamber-piece", and that Th
e Evening Standard stated the film's "best moments exhibit the bitchy tantrums seething benea
th the threesome's composed veneers".

-- Cluster 17 (size=80) --
• Lastly, Fiala mentions a critique towards philosophical anarchism of being ineffective (all
talk and thoughts) and in the meantime capitalism and bourgeois class remains strong.
• With this derivation, the name obtains a double meaning in the poem: when the hero is funct
ioning rightly, his men bring distress to the enemy, but when wrongly, his men get the grief
of war.
• The poem is in part about the misdirection of anger on the part of leadership.
• Achilles' wrath (μῆνις Ἀχιλλέως, mênis Achilléōs) is the central theme of the poem.
• It begins with Achilles' withdrawal from battle after being dishonoured by Agamemnon, the c
ommander of the Achaean forces.
```

Figure 18-1. Sample sentences from general-purpose clusters in selected bins.

```
-- Cluster 12 (size=39) --
• well i 'd love to get lunch later this week , but i 'm having lunch with my friend allison
today .
• i will not charge any of you with treason , but you need to understand why i love this woma
n .
• as the owner , i love showing guests around our operation .
• and even if i did propose before we dated much , i poured on the romance after that , belie
ve me .
• do n't tell me you were in love with him , sophie because i will simply destroy the man .
-- Cluster 1 (size=78) --
• as the others talked and laughed around him , aidan stared down at his son with such love i
n his eyes and adoration on his face that it cut a jagged hole in megan 's chest .
• my every nerve sizzles with anticipation as emerson unlocks our motel room love nest and wa
lks in before me .
• malik walked into the house and siri 's heart swelled with love for the man when she notice
d his eyes immediately looking for her , and then noticed his shoulders relaxing when he foun
d her in the front room , obviously waiting for him .
• with her feet barely on the floor , she felt zayn hold her gently against his side as he to
ssed the pillows off of her love seat .
• dharr walked forward , his experienced eyes noting other features in the lovely woman that
further piqued his interest .
```

Figure 18-2. Sample sentences from love-specific clusters in selected bins.

As shown in Figure 18, direct sentence inspection did not consistently reveal coherent themes, particularly in high-density clusters. To interpret what each cluster might represent, I applied lightweight topic modeling through word clouds. Word clouds are visual representations of the most frequent tokens in a given text, where word size reflects token frequency. Although this method does not capture deeper semantic relationships or context, it provides an accessible way to inspect the dominant lexical patterns in each cluster.

Figure 19-1. Word Clouds of General-purpose clusters (Left: cluster 31, Right: cluster 17).

Figure 19-2. Word Clouds of Love-specific clusters (Left: cluster 12, Right: cluster 1).

Figure 19 visualizes the word clouds for the same representative clusters shown in Figure 18. In the general-purpose dataset (Figure 19-1), the word cloud of cluster 31 is dominated by words like film, series, American, and score, indicating that this cluster contains sentences about film reviews or media coverage. Similarly, cluster 17 includes terms such as poem, men, and leadership, suggesting a topic centered on literary or classical content. These examples demonstrate that topic-relevant signals can emerge when sentence content is sufficiently diverse, and clusters are semantically grounded. By contrast, the love-specific dataset (Figure 19-2) exhibits word clouds for clusters 12 and 1 that are heavily dominated by frequent lexical items such as love, will, eyes, and friend. These terms reflect the thematic consistency of the dataset, which was filtered to contain only sentences involving "love" or "romance". As a result, the clustering outcome in this setting becomes less interpretable; many clusters appear visually similar in word clouds, and it is unclear what specific criteria distinguish them. This suggests that for thematically narrow corpora, word cloud-based inspection may offer limited insight into intra-cluster distribution, motivating the need for deeper analysis (e.g., hierarchical clustering).

To further explore the relationships between clusters beyond token frequency, I constructed a graph-based representation to approximate the hierarchical structure. Specifically, I computed pairwise cosine similarities between cluster centroids (using sentence embeddings) and connected cluster pairs whose similarity exceeded a fixed threshold (0.8). The resulting similarity graph, implemented in Figure 20, allows identification of central clusters (hubs) using network-based centrality metrics such as PageRank.

```
def identify_hub_clusters(embeddings, clusters, threshold=0.8):
    centers = compute_cluster_centers(embeddings, clusters)
    sim_matrix = cosine_similarity(centers)

    G = nx.Graph()
    n = sim_matrix.shape[0]
    for i in range(n):
        for j in range(i+1, n):
            if sim_matrix[i][j] >= threshold:
                G.add_edge(i, j, weight=sim_matrix[i][j])

    centrality = nx.pagerank(G)  # nx.degree_centrality(G), nx.pagerank(G), nx.betweenness_centrality(G)
    sorted_hubs = sorted(centrality.items(), key=lambda x: -x[1])[:5]  # top-5 center cluster

    print("Top Hub Clusters:")
    for cid, score in sorted_hubs:
        print(f"Cluster {cid}: Centrality Score = {score:.3f}")

    return G, sorted_hubs
```

Figure 20. Code for building a cluster similarity hierarchical graph using cosine similarity and PageRank.

While this method offers a rough sketch of higher-order cluster relationships, the visualization shown in Figure 21, based on general-purpose embeddings, reveals a structural limitation. Despite the constructed graph, the resulting topology appears densely interconnected, with no clear hub-and-spoke patterns or tree-like structures. Most clusters maintain a uniform degree of connectivity, suggesting that this algorithmic method alone is insufficient for uncovering meaningful parent-child relationships or layered hierarchies.

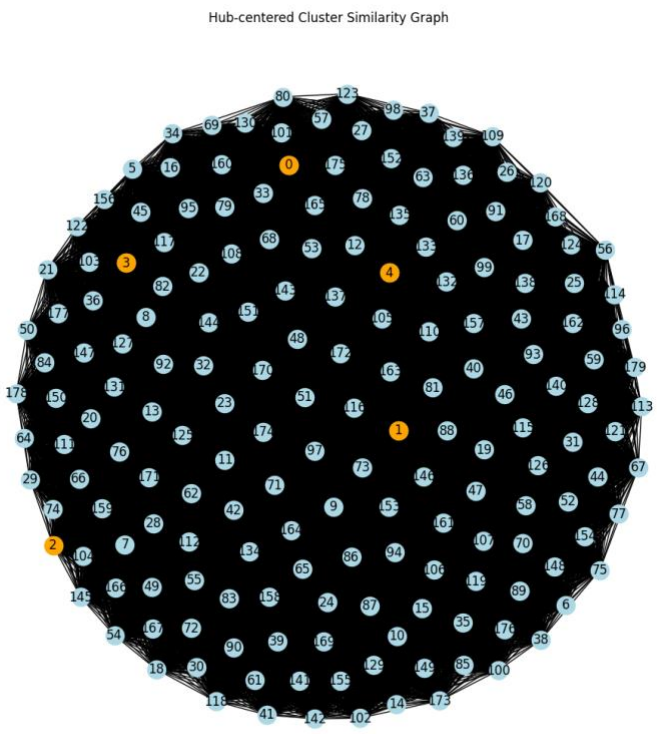


Figure 21. Inter-cluster similarity graph for general-purpose embeddings.
Orange nodes mark high-centrality clusters.

These findings highlight a limitation of my current approach: while histogram and topic modeling allow me to inspect within-cluster structure, they do not explain how clusters themselves relate to each other at a higher level. The absence of clear inter-cluster organization in the similarity graph suggests that a multi-level clustering approach, such as recursively clustering clusters (i.e., cluster-of-clusters), may be necessary to uncover deeper semantic hierarchies.

5. Evaluating Clustering Reliability: What is the Right Cluster?

While previous chapters explored how different clustering methods partition sentence embeddings and how these clusters reflect latent structure, this chapter addresses a more fundamental question: Which clustering method is correct? In other words, how should we evaluate clustering reliability and trust the resulting clusters?

To answer this, I conducted a comparative evaluation across five clustering strategies: Greedy, Graph-based, DBSCAN, K-Means, and GPT-based next-token prediction. All experiments used a general-purpose Wikipedia dataset consisting of 1,000 sentences. BERT embeddings were extracted from both final and intermediate layers for clustering, except in the GPT-based case, which uses token prediction outputs rather than embeddings.

To quantify clustering reliability, I use the silhouette score, which evaluates both:

- **Cohesion**: How close a point is to others within its own cluster
- **Separation**: How far a point is from points in other clusters

The score ranges from:

- **+1.0**: Perfect, well-separated clustering
- **0.0**: Overlapping clusters
- **< 0.0**: Poor clustering; points are closer to other clusters than their own

The following code snippet (Figure 22) shows how the silhouette score was computed using cosine distance:

```python
from sklearn.metrics import silhouette_score

def evaluate_clustering(embeddings, labels):
    if len(set(labels)) > 1 and -1 not in set(labels):  # avoid noise only or one cluster
        score = silhouette_score(embeddings, labels, metric="cosine")
    else:
        score = -1
    return score
```

Figure 22. Evaluation using Silhouette Score

For DBSCAN, I performed threshold sweeps to evaluate performance under different density assumptions (Figure 23). For K-Means, I used the elbow method to select optimal cluster counts, like in previous chapters. Both tuning procedures ensured fair evaluation across different clustering paradigms.

```python
def tune_dbscan(embeddings, threshold_list):
    results = []
    for thres in threshold_list:
        eps = 1 - thres
        dbscan = DBSCAN(eps=eps, min_samples=2, metric="cosine").fit(embeddings)
        labels = dbscan.labels_
        n_clusters = len(set(labels)) - (1 if -1 in labels else 0)
        n_noise = list(labels).count(-1)
        results.append((thres, n_clusters, n_noise))
    return results
```

```python
thresholds = [0.7, 0.75, 0.8, 0.85, 0.9]

final_dbscan_results = tune_dbscan(final_embeddings, thresholds)
inter_dbscan_results = tune_dbscan(intermediate_embeddings, thresholds)

print("FINAL EMBEDDINGS")
for thres, n_clusters, n_noise in final_dbscan_results:
    print(f"Threshold: {thres}, #Clusters: {n_clusters}, Noise Points: {n_noise}")

print("INTERMEDIATE EMBEDDINGS")
for thres, n_clusters, n_noise in inter_dbscan_results:
    print(f"Threshold: {thres}, #Clusters: {n_clusters}, Noise Points: {n_noise}")
```

Figure 23. Hyperparameter tuning of DBSCAN.

The summary of the silhouette score for each clustering method is shown in Table 5 below:

Clustering Method	Final Layer Silhouette	Intermediate Layer Silhouette
Greedy Algorithm	-0.041	-0.004
Graph-based Grouping	0.096	0.411
DBSCAN	-1.000	-1.000
K-Means Clustering	0.050	0.052
Next token prediction using GPT-2	-0.5086	-

Table 5. Silhouette Scores for Each Clustering Method.

Despite initial expectations that the Greedy algorithm might yield intuitive clusters due to its simplicity and direct reliance on local similarity, the results reveal otherwise. The final-layer Greedy clustering severely over-segments the space, producing numerous small and incoherent clusters. Even the intermediate-layer Greedy clustering improves only marginally (-0.004), which remains insufficient.

K-Means, with hyperparameter tuning via the elbow method, performed slightly better (~0.05), but its silhouette scores also still indicate weak internal cohesion and separation. The optimal cluster count (k = 220-260) is relatively large compared to the dataset size (1,000), implying that the model is over-segmenting and potentially capturing noise or fine-grained distinctions that do not correspond to meaningful categories.

DBSCAN, on the other hand, failed to produce meaningful clusters in all tested thresholds (epsilon). Its silhouette score was -1.0 across the board, indicating either total collapse (all samples in one cluster) or formation of structurally broken micro-clusters. This is likely due to a mismatch between DBSCAN's assumptions (e.g., density thresholds) and the high-dimensional, cosine-similarity-based nature of BERT embeddings. In such spaces, distance metrics become unreliable, often leading to unstable or degenerate clustering outcomes.

Next-token prediction-based clustering using GPT-2, while conceptually different from similarity-based methods, also performs poorly (-0.5086). This suggests that identical next-token predictions do not reliably reflect semantic similarity in embedding space. Many samples grouped by the same predicted token are actually closer to samples from other clusters. This is likely due to the generative nature of GPT-2, which may assign the same next token ("the", "a") across semantically diverse contexts. Furthermore, such prediction-based grouping results in many small clusters centered on frequent words, exacerbating the silhouette score via poor separation and size imbalance.

Interestingly, graph-based clustering using intermediate-layer BERT embeddings yields the highest silhouette score (0.411), far outperforming all other methods. At first glance, this suggests that intermediate representations encode semantically coherent structures. However, this result merits careful interpretation. Upon inspecting the graph, most nodes were found to be connected to a single dominant hub. This star-shaped structure would boost the silhouette score by improving cohesion since many nodes are assigned to a densely connected component while minimizing inter-cluster proximity. Additionally, the high score may partially reflect a hidden hierarchical structure within clusters; intermediate representations tend to encode overlapping or fine-grained semantic patterns, which the silhouette metric does not fully account for.

Thus, while the intermediate-layer graph-based approach appears numerically superior, this does not necessarily imply semantically superior clusters. In fact, the presence of latent substructure within these clusters, unaddressed by silhouette scoring, suggests that further fine-grained analysis is still needed (e.g., hierarchical sub-clustering or topic modeling).

These evaluations collectively suggest an unexpected outcome: none of the clustering methods tested yield consistently high-quality, semantically coherent groupings when applied to sentence embeddings from the current BERT model. This observation invites a critical reflection on the validity of earlier analyses (e.g., Zipfian regression, context vector analysis across layer-wise and domain-wise) that built upon these clusters. Rather than invalidating those findings, this result highlights a broader challenge: the potential limitations of the embedding representations themselves.

One plausible explanation lies in the scale and expressiveness of the underlying embedding model. All experiments in this report rely on relatively compact BERT architectures. It is possible that these models do not encode sufficiently disentangled or clusterable semantic representations, especially in high-dimensional, unsupervised settings. The clustering methods may, in turn, struggle not because of inherent flaws in their design, but because they operate on a noisy or entangled representation space. This opens an important direction for future work: evaluating clustering reliability not in isolation, but in conjunction with embedding quality. Larger-scale or more specialized embedding modes may yield more structured representations that enable clearer cluster separation. Whether this would ultimately yield more interpretable clusters is still uncertain, but it remains a promising path for further investigation.

6. Conclusions and Future Works

This report presents a multi-stage investigation into how sentence embeddings encode semantic information and how different clustering methods extract, reveal, or distort this structure. The five chapters explore global scaling behaviors (Chapter 1), representational variation across layers and domains (Chapter 2), the effects of specific clustering algorithms (Chapter 3), intra-cluster structure (Chapter 4), and quantitative evaluation of clustering reliability (Chapter 5).

In Chapter 1, I examined whether sentence embedding clusters in embedding space follow Zipfian patterns in their size distribution. Using the greedy and K-Means clustering on a general-purpose dataset, I observed approximate Zipf-like behavior in cluster size rankings. However, the Zipf slope was sensitive to both clustering methods and preprocessing, complicating interpretation. These results suggest a possible scaling law in semantic grouping but highlight the need for normalization and methodological care.

Chapter 2 extended this investigation by analyzing how sentence representations vary across embedding layers and domains. It showed that intermediate layers tend to encode more structured semantic groupings than final layers, and that domain specificity amplifies this effect. These results emphasize the importance of both layer selection and data semantics when interpreting embedding-based structure.

Chapter 3 compared greedy and K-Means clustering with a token-prediction-based approach. The results showed that greedy clustering performs robustly, especially at intermediate layers, while K-Means tends to underrepresent finer semantics due to its centroid-driven structure. In contrast, token prediction-based grouping failed to reflect meaningful semantic structure, instead exposing the extrapolation limits of generative models under synthetic inputs.

Chapter 4 explored the internal structure of clusters through a histogram-based binning strategy. Sentence samples and word cloud visualizations revealed that clusters with similar sizes could vary significantly in semantic coherence. Some clusters displayed strong topical signals, while others were lexically noisy. To further

analyze structural patterns, I constructed a cluster similarity graph using pairwise cosine distances between cluster centroids, but no clear hierarchical or hub-like structure emerged. These results highlighted the limits of flat clustering and motivated a hierarchical approach.

Chapter 5 evaluated clustering reliability using silhouette scores across five methods. While graph-based clustering using intermediate-layer embeddings yielded the highest score (0.411), this was likely inflated by hub-centric structures. All other methods, including Greedy, DBSCAN, K-Means, and GPT-based, showed low or negative silhouette scores, suggesting weak separation and frequent misassignment. These findings underscore the importance of both the algorithm and the underlying representation in clustering performance.

From these findings, three promising future research directions emerge:

First, in terms of hierarchical clustering via multi-stage or binning strategies, the analyses in Chapters 2-4 suggest that a single clustering pass is insufficient to isolate semantically pure clusters. A two-stage approach, beginning with coarse segmentation and followed by finer clustering within high-density regions, can reduce intra-cluster noise and reveal hierarchical structure. Topic modeling at each level could further enhance interpretability.

Second, for ontology-grounded multi-stage word embedding analysis, while this report focuses on sentence embeddings, similar techniques can be applied to word embeddings using ontology datasets. Specifically, ontologies exhibit natural parent-child hierarchies, offering a ground truth for evaluating quality. Multi-stage clustering can be applied to parent and child categories, enabling: (1) For clustering interpretability, given that we know the correct hierarchy, we can evaluate how close a multi-way clustering without labels approximates it; (2) For semantic consistency, by measuring the cosine similarity between a cluster centroid and its associate category embedding or mean of in-cluster words, we can probe the cohesion and topic alignment within each cluster.

Lastly, in the context of exploring the relation between model scale and clustering quality. So far, all analyses used sentence embeddings from relatively compact BERT models. The low silhouette scores observed may reflect representational limitations rather than algorithmic flaws. Future work could fix the clustering method and systematically vary the embedding model to investigate how model scale influences cluster density, cohesion, and separability. This could shed light on what constitutes a “good” embedding for unsupervised structure discovery.